SIMATS ENGINEERING
ASSESSMENT TOOL 1 - HIGH
(ANALYTICAL PROBLEM SOLVING)

COURSE CODE: ITA04
COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO3: *Construct and manipulate data frames using reshaping, merging, and file I/O operations in R to prepare structured datasets for analysis. (BL3, BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 10 **Total Marks: 100**

ANNEXURE A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I **A. Midhuna (192521302)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.
A. Midhuna
Signature of the student:

1. Making Data Frames

You receive raw sensor log vectors (sensor_id as character, voltage as numeric, status as logical) of unequal lengths (\(N_1 = 120\), \(N_2 = 119\), \(N_3 = 120\)) from an embedded diagnostic stream. If you pass these directly into the data.frame() constructor, R triggers a recycling-rule exception or silent mismatch warning. Analyze the structural failure mechanism of R's internal memory allocation during this coercion, and write an R routine that programmatically inspects and rectifies vector asymmetry by padding the shortest vector with NA *only after* generating a diagnostic discrepancy report detailing the missing index positions.

2. Working with Data Frames

An engineering telemetry frame contains a continuous numeric column Temperature with intermittent anomalies coded as literal character strings "ERR_99" instead of standard numeric NA values. Differentiate the impact this has on vectorized operations like mean(df$Temperature, na.rm = TRUE) (which fails due to character coercion) versus indexing subsets. Construct an analytical data-wrangling script that isolates non-numeric string tokens, logs their row indices and exact token frequency, replaces them with explicit NA, and safely type-casts the column back to double precision without truncating valid numeric records.

3. Data Reshaping

A structural strain matrix records measurements across 4 spatial coordinate sensors over 3time steps, but the data is spread across multi-indexed columns (SensorA_X, SensorA_Y, etc.). Deconstruct why maintaining this wide format creates a dimensional restriction for spatial regression models requiring independent observation rows. Use base R or reshape mechanics to transpose and restructure the coordinate attributes into a cohesive time-series layout without losing row-to-observation mapping integrity.

4. Melting and Casting of Data

Using the reshape2 package, you apply melt() on a multi-sensor performance matrix. However, duplicate ID combinations exist for identical timestamps due to dual-bitrate sampling redundancy. If you execute dcast() directly on this molten data frame without an aggregation function, R throws a structural ambiguity error. Analyze why multiple data points mapping to a single cell require an explicit reduction function (fun.aggregate), and write a dcast command that computes both the mean and variance simultaneously to preserve variance analysis during the wide-format conversion.

5. Merging Data Frames

You are tasked with fusing df_control (keyed by SystemID and Timestamp) and df_telemetry (keyed by SystemID and Timestamp). Both tables contain a column named Status, which results in conflicting .x and .y suffixes upon a standard inner join via merge(). Furthermore, time stamps in the two tables drift by up to 2 seconds due to

asynchronous clock skews, causing an inner join on exact values to drop 40% of records. Analyze how this data loss biases the system reliability estimation, and design a custom interval-matching strategy or pre-processing rounding mechanism before executing an outer merge that preserves unmatched orphan error logs.

Making Data Frames

Q1. Design a data frame structure for a smart-city traffic monitoring system and justify the selection of attributes.

Answer:

Coding:

```
 traffic <- data. Frame (
 Sensor_ ID = c (101, 102, 103),
 Location = c ("Gandhipuram", "Peelamedu", "RS Puram"),
 Vehicle_Count = c (250, 180, 210),
 Average_Speed = c (40.5, 35.2, 38.7),
 Signal_Status = c (TRUE, FALSE, TRUE)
)
print(traffic)
```

Output

	Sensor_ID	Location	Vehicle_Count	Average_Speed	Signal_Status
1	101	Gandhipuram	250	40.5	TRUE
2	102	Peelamedu	180	35.2	FALSE
3	103	RS Puram	210	38.7	TRUE

Q2. Evaluate two alternative data frame designs for storing IoT sensor data and recommend the most scalable model.

Answer:
Coding:

```
# Wide format
wide <- data. Frame (
 Time = c ("10:00", "10:01"),
Temperature = c (30.5, 31.0),
 Humidity = c (65, 67)
)
Print ("Wide Format:")
```

```
print(wide)

# Long format
long <- data. frame (
Time = c ("10:00", "10:00", "10:01", "10:01"),
Sensor_Type = c ("Temperature", "Humidity",
 "Temperature", "Humidity"),
 Value = c (30.5, 65, 31.0, 67)
)
Print ("Long Format:")
print(long)
# Recommendation
Print ("Long format is more scalable for IoT sensor data.")
```

```
[1] "Wide Format:"
   Time Temperature Humidity
1 10:00        30.5       65
2 10:01        31.0       67
[1] "Long Format:"
   Time Sensor_Type Value
1 10:00 Temperature  30.5
2 10:00    Humidity  65.0
3 10:01 Temperature  31.0
4 10:01    Humidity  67.0
[1] "Long format is more scalable for IoT sensor data."
```

Q3. Analyze the consequences of assigning incorrect data types while creating a data frame for healthcare records.

Coding:

```
# Create healthcare data with incorrect data types
patient <- data. frame (
 Patient_ID = c ("001", "002", "003"),
Age = c ("25", "40", "35"),
 Temperature = c ("37.2", "38.1", "36.8")
)

Print ("Original Data:")
print(patient)
str(patient)

# Convert to correct data types
patient$Age <- as.numeric(patient$Age)
patient$Temperature <- as.numeric(patient$Temperature)
```

```
print ("Corrected Data:")
print(patient)
str(patient)

# Calculate average age and temperature
print(mean(patient$Age))
print(mean(patient$Temperature))
```

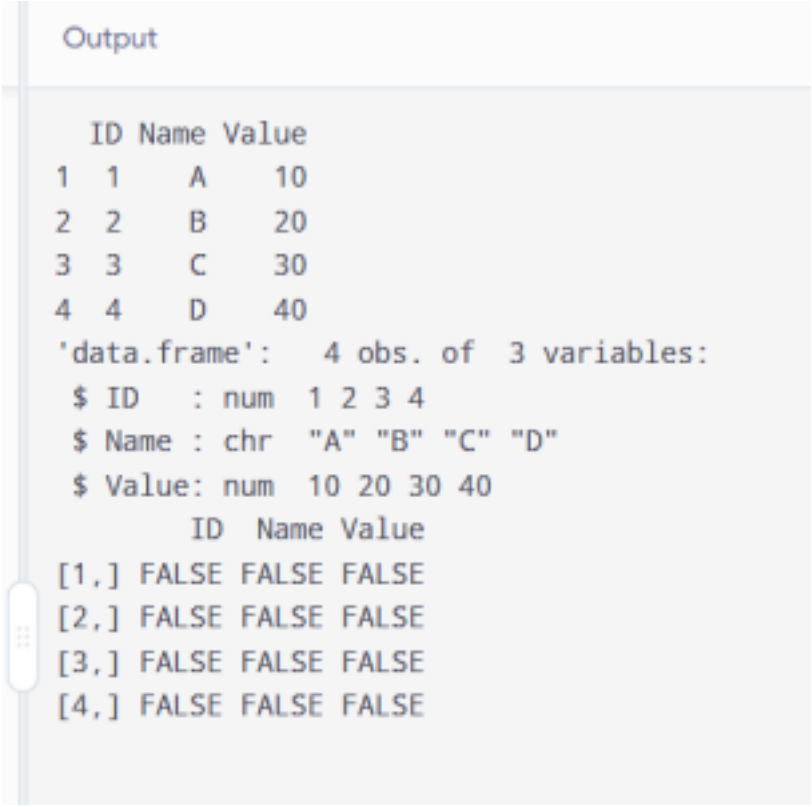
```
[1] "Original Data:"
  Patient_ID Age Temperature
1        001  25        37.2
2        002  40        38.1
3        003  35        36.8
'data.frame':   3 obs. of  3 variables:
 $ Patient_ID : chr  "001" "002" "003"
 $ Age        : chr  "25" "40" "35"
 $ Temperature: chr  "37.2" "38.1" "36.8"
[1] "Corrected Data:"
  Patient_ID Age Temperature
1        001  25        37.2
2        002  40        38.1
3        003  35        36.8
'data.frame':   3 obs. of  3 variables:
 $ Patient_ID : chr  "001" "002" "003"
 $ Age        : num  25 40 35
 $ Temperature: num  37.2 38.1 36.8
[1] 33.33333
[1] 37.36667
```

Q4. Create a strategy for constructing a data frame from multiple heterogeneous data sources while preserving consistency.

Coding:

```
# Create two data sources
data1 <- data. frame (
ID = c (1, 2),
Name = c ("A", "B"),
Value = c (10, 20)
)
data2 <- data. frame (
ID = c (3, 4),
Name = c ("C", "D"),
Value = c (30, 40)
```

)
Combine data frames
combined <- rbind(data1, data2)
Display the combined data
print(combined)
Check structure and missing values
str(combined)
print(is.na(combined))



Q5. Assess the efficiency of creating a data frame from large vectors versus importing directly from files.

Coding:

Create data frame from large
vectors id <- 1:100000
value <- runif(100000)
df1 <- data. frame (ID = id, Value
= value) cat ("Data Frame from
Vectors:\n")
print(head(df1))
Create a temporary CSV file
file_path <- tempfile(fileext = ".csv")
write.csv (df1, file_path, row.names =
FALSE) # Import data from file
df2 <- read.csv(file_path)
cat ("Data Frame Imported from
File:\n") print(head(df2))

cat ("Total rows:", nrow(df2), "\n")

```
Data Frame from Vectors:
  ID     Value
1  1 0.5596953
2  2 0.8385617
3  3 0.3027322
4  4 0.4219777
5  5 0.9519991
6  6 0.6096424
Data Frame Imported from File:
  ID     Value
1  1 0.5596953
2  2 0.8385617
3  3 0.3027322
4  4 0.4219777
5  5 0.9519991
6  6 0.6096424
Total rows: 100000
```

Working with Data Frames

Q1. Analyze a scenario where filtering operations on a large data frame are producing unexpected results. Determine the root cause.

Coding:

```
df <- data.frame(Temperature = c(25, 30, NA, 35))
result <- df[df$Temperature > 28,]
print(result)
```

```
[1] 30 NA 35
```

Q2. Evaluate different methods for sorting a million-row data frame and justify the most efficient approach.

Coding:

```
df <- data.frame(
  Temperature = c(35, 25, 40, 20, 30)
)
df <- df[order(df$Temperature), ]
print(df)
```

```
[1] 20 25 30 35 40
```

Q3. A data analyst performs multiple transformations on a data frame. Analyze how errors can propagate through the workflow.

Coding:

```
df <- data.frame(
  Temperature = c(25, 30, NA, 35)
)
df$Temperature <- as.numeric(df$Temperature)
stopifnot(is.numeric(df$Temperature))
summary(df$Temperature)
sum(is.na(df$Temperature))
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   25.0    27.5    30.0    30.0    32.5    35.0       1
[1] 1
```

Q4. Compare indexing and logical subsetting techniques for extracting data and evaluate their performance trade-offs.

Coding:

```
df <- data.frame(
  ID = 1:5,
  Temperature = c(25, 35, 30, 40, 20)
)
# Indexing
print(df[1:3, ])
# Logical subsetting
print(df[df$Temperature > 30, ])
```

```
  ID Temperature
1  1          25
2  2          35
3  3          30
  ID Temperature
2  2          35
4  4          40
```

Q5. Design a workflow to monitor and validate modifications made to a shared data frame by multiple users.

Coding:

```
# Create a data frame
df <- data.frame(
```

```
  ID = 1:4,
  Temperature = c(25, 30, 35, 40)
)
# Save original data
original_df <- df
# Create an audit log
audit_log <- data.frame(
  User = character(),
  Change = character(),
  stringsAsFactors = FALSE
)
# Modify the data
df$Temperature[1] <- 28
# Record the modification
audit_log <- rbind(
  audit_log,
  data.frame(
    User = "User1",
    Change = "Updated Temperature for ID 1"
  )
)
# Validate the data
stopifnot(is.numeric(df$Temperature))
stopifnot(!anyDuplicated(df$ID))
stopifnot(all(df$Temperature > 0))
# Display updated data and audit log
print(df)
print(audit_log)
```

```
  ID Temperature
1  1          28
2  2          30
3  3          35
4  4          40
   User                       Change
1 User1 Updated Temperature for ID 1
```

Data Reshaping

Q1. Analyze a business dataset before and after reshaping. Evaluate how the transformation affects data interpretation.

Coding:

```
# Create business dataset in wide format
df <- data.frame(
  Product = c ("Laptop", "Mobile"),
  January = c (50000, 30000),
  February = c (60000, 40000),
```

```
  March = c (55000, 35000)
)
Cat ("Before Reshaping:\n")
print(df)
# Reshape from wide to long format
library(tidyr)
long_df <- pivot_longer(
  df,
  cols = January:March,
  names_to = "Month",
  values_to = "Sales"
)
Cat ("After Reshaping:\n")
print(long_df)
```

```
Before Reshaping:
  Product January February March
1  Laptop   50000    60000 55000
2  Mobile   30000    40000 35000
```

Q2. A dataset must be reshaped for machine learning. Determine which structure is most suitable and justify your decision.

Coding:

```
install.packages("tidyr")
library(tidyr)
df <- data.frame(
  ID = c(1, 2),
  Math = c(80, 90),
  Science = c(85, 95)
)
long_df <- pivot_longer(
  df,
  cols = c("Math", "Science"),
  names_to = "Subject",
  values_to = "Marks"
)
print(long_df)
```

```
Installing package into ‘/usr/local/lib/R/site-library’
(as ‘lib’ is unspecified)
```

Q3. Evaluate the risks of reshaping data incorrectly in a healthcare analytics application.

Coding:

```
df <- data.frame(
  Patient_ID = c(101, 102, 103),
  BP = c(120, 140, 130),
  Sugar = c(90, 150, 110)
```

```
)
long_df <- data.frame(
  Patient_ID = rep(df$Patient_ID, 2),
  Test = rep(c("BP", "Sugar"), each = 3),
  Value = c(df$BP, df$Sugar)
)
print(long_df)
print(anyNA(long_df))
print(anyDuplicated(long_df))
```

```
  Patient_ID  Test Value
1        101    BP   120
2        102    BP   140
3        103    BP   130
4        101 Sugar    90
5        102 Sugar   150
6        103 Sugar   110
[1] FALSE
[1] 0
```

Q4. Compare wide and long data formats for time-series analysis and recommend the better approach for forecasting.

Coding:

```
df <- data.frame(
  Year = c(2024, 2025),
  Sales_Jan = c(100, 120),
  Sales_Feb = c(150, 170)
)
long_df <- data.frame(
  Year = rep(df$Year, 2),
  Month = rep(c("Jan", "Feb"), each = 2),
  Sales = c(df$Sales_Jan, df$Sales_Feb)
)
print(long_df)
```

```
  Year Month Sales
1 2024   Jan   100
2 2025   Jan   120
3 2024   Feb   150
4 2025   Feb   170
```

Q5. Design a reshaping strategy for integrating survey responses collected across multiple years.

Coding:

```
year1 <- data.frame(ID = c(1, 2), Year = 2024,
```

```
 Q1 = c(5, 4), Q2 = c(3, 5))
year2 <- data.frame(ID = c(3, 4), Year = 2025,
  Q1 = c(4, 5), Q2 = c(5, 3))
# Combine survey data
df <- rbind(year1, year2)
# Reshape into long format
long_df <- data.frame(
  ID = rep(df$ID, 2),
  Year = rep(df$Year, 2),
  Question = rep(c("Q1", "Q2"), each = nrow(df)),
  Response = c(df$Q1, df$Q2)
)
print(long_df)
```

```
  ID Year Question Response
1  1 2024       Q1        5
2  2 2024       Q1        4
3  3 2025       Q1        4
4  4 2025       Q1        5
5  1 2024       Q2        3
6  2 2024       Q2        5
7  3 2025       Q2        5
8  4 2025       Q2        3
```

Melting and Casting of Data

Q1. Analyze a dataset where repeated measurements are stored in multiple columns. Determine how melting improves analysis.

Coding:

```
df <- data.frame(
  ID = c(1, 2, 3),
  Test1 = c(70, 80, 90),
  Test2 = c(75, 85, 95)
)
print("Before Melting:")
print(df)
long_df <- data.frame(
  ID = rep(df$ID, 2),
  Test = rep(c("Test1", "Test2"), each = 3),
  Marks = c(df$Test1, df$Test2)
)
Print ("After Melting:")
print(long_df)
```

```
[1] "Before Melting:"
  ID Test1 Test2
1  1    70    75
2  2    80    85
3  3    90    95
[1] "After Melting:"
  ID  Test Marks
1  1 Test1    70
2  2 Test1    80
3  3 Test1    90
4  1 Test2    75
5  2 Test2    85
6  3 Test2    95
```

Q2. Evaluate the impact of casting data into different summary structures for business reporting.

Coding:

```
df <- data.frame(
  Product = c("Laptop", "Mobile", "Laptop", "Mobile"),
  Year = c(2024, 2024, 2025, 2025),
  Sales = c(50000, 30000, 60000, 40000)
)
print("Original Data:")
print(df)
# Cast data into summary format
summary_df <- xtabs(Sales ~ Product + Year, data = df)
print("Summary Data:")
print(summary_df)
```

```
[1] "Original Data:"
  Product Year Sales
1  Laptop 2024 50000
2  Mobile 2024 30000
3  Laptop 2025 60000
4  Mobile 2025 40000
[1] "Summary Data:"
        Year
Product   2024  2025
  Laptop 50000 60000
  Mobile 30000 40000
```

Q3. Compare two casting approaches that produce different aggregation results and explain the reasons.

Coding:

```
df <- data.frame(
```

```
  Product = c("A", "A", "B", "B"),
  Sales = c(100, 200, 150, 250)
)
# Approach 1: Sum aggregation
sum_df <- aggregate(Sales ~ Product, data = df, sum)
print(sum_df)
# Approach 2: Mean aggregation
mean_df <- aggregate(Sales ~ Product, data = df, mean)
print(mean_df)
```

```
  Product Sales
1       A   300
2       B   400
  Product Sales
1       A   150
2       B   200
```

Q4. Design a workflow using melting and casting to prepare data for predictive modeling.

Coding:

```
df <- data.frame(
  ID = 1:3,
  Math = c(80, 90, 70),
  Science = c(85, 95, 75)
)
long_df <- reshape(
  df,
  varying = c("Math", "Science"),
  v.names = "Marks",
  timevar = "Subject",
  times = c("Math", "Science"),
  direction = "long"
)
print(long_df)
wide_df <- reshape(
  long_df,
  idvar = "ID",
  timevar = "Subject",
  direction = "wide"
)
print(wide_df)
```

```
          ID Subject Marks id
1.Math     1    Math    80  1
2.Math     2    Math    90  2
3.Math     3    Math    70  3
1.Science  1 Science    85  1
2.Science  2 Science    95  2
3.Science  3 Science    75  3
       ID Marks.Math id.Math Marks.Science id.Science
1.Math  1         80       1            85          1
2.Math  2         90       2            95          2
3.Math  3         70       3            75          3
```

Q5. Assess the challenges of applying melt and cast operations on large-scale industrial datasets.

Coding:

```
df <- data.frame(ID=1:3, A=c(80,90,85), B=c(70,75,95))
long <- reshape(df, direction="long",
            varying=c("A","B"), v.names="Marks",
            timevar="Machine", times=c("A","B"))
print(long)
wide <- reshape(long, idvar="ID",
            timevar="Machine", direction="wide")
print(wide)
```

```
    ID Machine Marks id
1.A  1       A    80  1
2.A  2       A    90  2
3.A  3       A    85  3
1.B  1       B    70  1
2.B  2       B    75  2
3.B  3       B    95  3
    ID Marks.A id.A Marks.B id.B
1.A  1      80    1      70    1
2.A  2      90    2      75    2
3.A  3      85    3      95    3
```

Merging Data Frames

Q1. Analyze the consequences of performing an incorrect join operation between two financial datasets.

Coding:

```
df1 <- data.frame(ID = c(1, 2, 3), Amount = c(100, 200, 300))
df2 <- data.frame(ID = c(1, 2, 4), Profit = c(10, 20, 40))
```

```
# Incorrect join
result <- merge(df1, df2, by = "ID", all = TRUE)
print(result)
```

```
  ID Amount Profit
1  1    100     10
2  2    200     20
3  3    300     NA
4  4     NA     40
```

Q2. Evaluate inner, outer, left, and right joins for integrating customer and transaction data.

Coding:

```
customers <- data.frame(ID = c(1, 2, 3), Name = c("A", "B", "C"))
transactions <- data.frame(ID = c(1, 2, 4), Amount = c(100, 200, 400))
# Inner Join
Print (merge (customers, transactions, by = "ID"))
# Outer Join
Print (merge (customers, transactions, by = "ID", all = TRUE))
# Left Join
Print (merge (customers, transactions, by = "ID", all.x = TRUE))
# Right Join
Print (merge (customers, transactions, by = "ID", all.y = TRUE))
```

```
  ID Name Amount
1  1    A    100
2  2    B    200
  ID Name Amount
1  1    A    100
2  2    B    200
3  3    C     NA
4  4 <NA>    400
  ID Name Amount
1  1    A    100
2  2    B    200
3  3    C     NA
  ID Name Amount
1  1    A    100
2  2    B    200
3  4 <NA>    400
```

Q3. A merged data frame contains unexpected duplicate rows. Investigate the possible causes and propose solutions.

Coding:

```
df1 <- data.frame(ID = c(1, 2, 2, 3), Name = c("A", "B", "B", "C"))
df2 <- data.frame(ID = c(1, 2, 2, 4), Amount = c(100, 200, 200, 400))
```

```
# Check duplicate rows
print(duplicated(df1))
print(duplicated(df2))
# Remove duplicate rows
df1 <- unique(df1)
df2 <- unique(df2)
# Merge data frames
result <- merge (df1, df2, by = "ID")
print(result)
```

```
[1] FALSE FALSE  TRUE FALSE
[1] FALSE FALSE  TRUE FALSE
  ID Name Amount
1  1    A    100
2  2    B    200
```

Q4. Design a strategy to merge data frames from multiple departments while ensuring data consistency.

Coding:

```
# Department datasets
hr <- data.frame(ID = c(1, 2), Name = c("A", "B"))
finance <- data.frame(ID = c(1, 2), Salary = c(20000, 30000))
sales <- data.frame(ID = c(1, 2), Sales = c(5000, 8000))
# Merge datasets
merged_data <- merge(hr, finance, by = "ID")
merged_data <- merge(merged_data, sales, by = "ID")
# Check missing values and duplicates
print(merged_data)
print(anyDuplicated(merged_data$ID))
print(is.na(merged_data))
# Remove duplicate rows
merged_data <- unique(merged_data)
print(merged_data)
```

```
  ID Name Salary Sales
1  1    A  20000  5000
2  2    B  30000  8000
[1] 0
        ID  Name Salary Sales
[1,] FALSE FALSE  FALSE FALSE
[2,] FALSE FALSE  FALSE FALSE
  ID Name Salary Sales
1  1    A  20000  5000
2  2    B  30000  8000
```

Q5. Compare key-based merging and index-based merging techniques for large engineering datasets.

Coding:

```
# Key-based merging
df1 <- data.frame(ID = c(1, 2, 3), Value = c(10, 20, 30))
df2 <- data.frame(ID = c(1, 2, 3), Temp = c(25, 30, 35))
key_merge <- merge(df1, df2, by = "ID")
print(key_merge)
# Index-based merging
index_merge <- data.frame(df1, df2["Temp"])
print(index_merge)
```

```
  ID Value Temp
1  1    10   25
2  2    20   30
3  3    30   35
  ID Value Temp
1  1    10   25
2  2    20   30
3  3    30   35
```

Data Frames

Q1. Analyze a dataset of 50,000 student records stored in a data frame. Identify the most efficient method to detect duplicate entries and justify your approach.

Coding:

```
# Create sample student dataset
df <- data.frame(ID = c (1, 2, 3, 2, 4),
                 Name = c ("A", "B", "C", "B", "D"))
```

```
# Detect duplicate records
duplicates <- df[duplicated(df),]
print(duplicates)
# Count duplicate records
print(sum(duplicated(df)))
```

```
  ID Name
4  2    B
[1] 1
```

Q2. A data frame contains missing values across multiple columns. Evaluate different strategies for handling missing data and recommend the best solution.

Coding:

```
# Sample dataset
df <- data.frame(Name = c("A", "B", "C"),
                 Age = c(20, NA, 22),
                 Marks = c(80, 90, NA))
# Check missing values
print(is.na(df))
# Remove rows with missing values
df1 <- na.omit(df)
print(df1)
# Replace missing numeric values with column mean
df$Age[is.na(df$Age)] <- mean(df$Age, na.rm = TRUE)
df$Marks[is.na(df$Marks)] <- mean(df$Marks, na.rm = TRUE)
print(df)
```

```
      Name   Age Marks
[1,] FALSE FALSE FALSE
[2,] FALSE  TRUE FALSE
[3,] FALSE FALSE  TRUE
  Name Age Marks
1    A  20    80
  Name Age Marks
1    A  20    80
2    B  21    90
3    C  22    85
```

Q3. Given a sales data frame with inconsistent data types, analyze the impact on statistical calculations and propose corrective actions.

Coding:

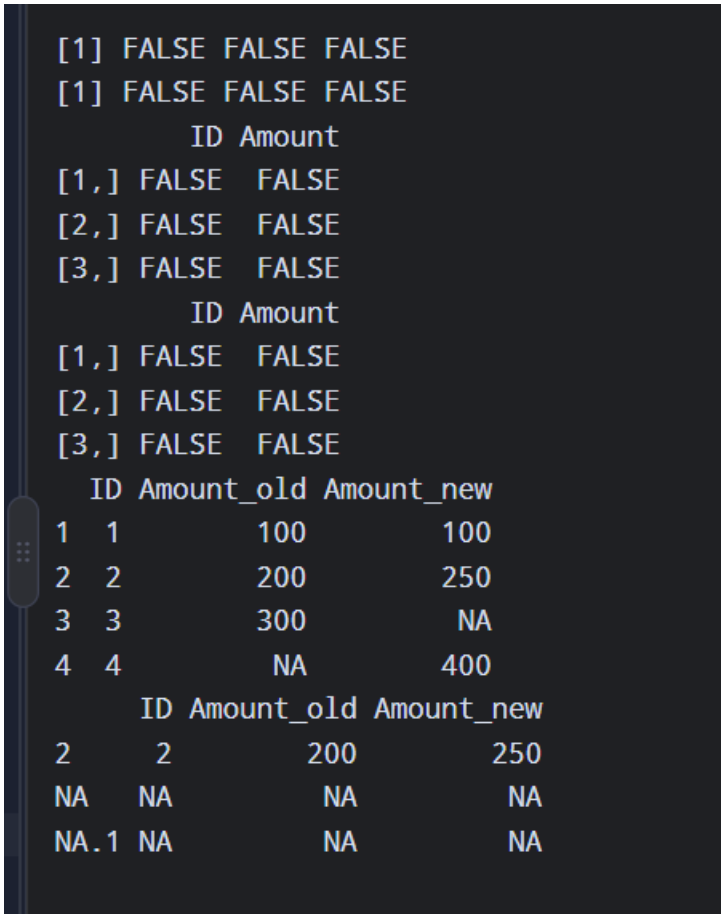
```
# Sales dataset with inconsistent data types
df <- data.frame(
  Product = c("A", "B", "C"),
  Sales = c("100", "200", "300")
)
# Convert Sales to numeric
df$Sales <- as.numeric(df$Sales)
# Perform statistical calculations
print(mean(df$Sales))
print(sum(df$Sales))
print(summary(df))
```

```
[1] 200
[1] 600
   Product             Sales
 Length:3           Min.   :100
 Class :character   1st Qu.:150
 Mode  :character   Median :200
                    Mean   :200
                    3rd Qu.:250
                    Max.   :300
```

Q4. Compare two large data frames containing customer transactions. Determine how to identify data integrity issues and evaluate their causes.

Coding:

```
# Sample transaction data
df1 <- data.frame(ID = c(1, 2, 3), Amount = c(100, 200, 300))
df2 <- data.frame(ID = c(1, 2, 4), Amount = c(100, 250, 400))
# Find duplicate records
print(duplicated(df1))
print(duplicated(df2))
# Find missing values
print(is.na(df1))
print(is.na(df2))
# Compare transactions
result <- merge(df1, df2, by = "ID", all = TRUE,
                suffixes = c("_old", "_new"))
print(result)
# Identify amount mismatches
print(result[result$Amount_old != result$Amount_new,])
```



Q5. A company reports conflicting results from the same data frame. Analyze possible reasons and design a validation framework.

Coding:

```
# Sample data
df <- data.frame(ID = c(1, 2, 3), Sales = c(100, 200, 300))
# Validation framework
print(str(df))
print(summary(df))
print(anyNA(df))
print(anyDuplicated(df$ID))
# Calculate total sales
total1 <- sum(df$Sales)
total2 <- sum(df$Sales)
# Validate results
if (total1 == total2) {
  print ("Validation Successful")
} else {
  Print ("Results Mismatch")
}
print(total1)
```

```
'data.frame':   3 obs. of  2 variables:
 $ ID   : num  1 2 3
 $ Sales: num  100 200 300
NULL
       ID          Sales
 Min.   :1.0   Min.   :100
 1st Qu.:1.5   1st Qu.:150
 Median :2.0   Median :200
 Mean   :2.0   Mean   :200
 3rd Qu.:2.5   3rd Qu.:250
 Max.   :3.0   Max.   :300
[1] FALSE
[1] 0
[1] "Validation Successful"
[1] 600
```

RUBRICS FOR CALCULATION

Class Test					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Data Frame Concepts	25	Demonstrates thorough understanding of Data Frame concepts and operations.	Demonstrates good understanding with minor omissions.	Demonstrates basic understanding with limited details.	Demonstrates poor or incorrect understanding of Data Frame concepts.
Understanding of Reshaping and Merging Techniques	25	Clearly explains reshaping and merging concepts with appropriate use cases.	Explains concepts correctly with minor gaps.	Provides partial understanding of reshaping or merging concepts.	Unable to explain reshaping or merging concepts correctly.
Knowledge of File I/O Operations	20	Accurately explains file handling	Explains most file handling	Provides limited explanation of file operations.	Demonstrates little understanding of

		functions and their usage.	concepts correctly.		file I/O operations.
Application to Data Analysis and Dataset Preparation	20	Effectively applies concepts to data preparation and analysis tasks.	Provides appropriate applications with adequate explanation.	Gives limited applications or examples.	Unable to relate concepts to practical applications.
Technical Accuracy and Completeness	10	Uses correct terminology and provides complete answers.	Mostly accurate with minor technical errors.	Some inaccuracies and incomplete responses.	Major technical errors and incomplete answers.